GUIA DE ATIVIDADE PRÁTICA — AULA 03

Persistência Relacional com Spring Data JPA, Hibernate & H2 Database

Projeto Evolutivo MyMovies | Prefix de Pacote: pt.rumos.mymovies

1. Resumo Teórico & Dicas de Arquitetura (Guia de Estudo)

Nas Aulas 01 e 02, construímos a nossa API REST mantendo a lista de filmes armazenada diretamente na memória RAM (**List<Movie>**). Embora funcional para testes rápidos, essa abordagem possui um problema crítico: **a volatilidade dos dados**. Sempre que a aplicação Spring Boot é reiniciada, todos os registos inseridos são perdidos. Na Aula 03, evoluímos a arquitetura para utilizar a persistência relacional com **Spring Data JPA, Hibernate e Banco H2**.

1.1. Por que a Entidade JPA NÃO pode ser um Record do Java 21?

Um dos pontos de maior confusão em Java moderno é entender onde usar **Records** e onde usar **Classes de Entidade JPA**:

- **Records (Java 21):** São estruturas imutáveis por design (todos os campos são final, não possuem getters/setters tradicionais e não podem ser estendidos). São ideais para DTOs (*MovieRequest, MovieResponse*).
- **Entidades JPA (@Entity):** A especificação Jakarta Persistence exige que a entidade seja uma classe Java tradicional (*class*) com um construtor padrão sem argumentos (*no-args constructor*) e atributos mutáveis. Isso ocorre porque o Hibernate precisa de instanciar a classe via reflexão e criar Proxies dinâmicos de sub-classe para suporte a Lazy Loading.

1.2. Mapeamento de Entidade Jakarta Persistence

As principais anotações para transformar uma classe POJO numa tabela de banco de dados relacional são:

Anotação JPA	Descrição e Função Prática
`@Entity`	Informa ao JPA/Hibernate que a classe é mapeada para uma tabela do banco de dados.
`@Table(name = "movies")`	Define explicitamente o nome da tabela no SGBD (evita conflitos com palavras reservadas).
`@Id + @GeneratedValue`	@Id marca a Chave Primária (PK). @GeneratedValue(strategy = IDENTITY) delega ao banco o autoincremento.
`@Column(name = "...",`	Mapeia o nome da coluna física na tabela e define restrições como NOT NULL

`nullable = false)` e tamanho máximo.

1.3. Modo de Persistência H2: Ficheiro Local vs. Em Memória

Por padrão, o banco de dados H2 funciona **em memória** (*jdbc:h2:mem:testdb*), apagando os dados ao desligar o Spring Boot. Para podermos realizar a **Prova do Restart** no laboratório, configuramos o H2 no modo **Ficheiro Local (File Mode)**:

```
# Configuração H2 em Ficheiro Persistente (./data/mymovies.mv.db)
spring.datasource.url=jdbc:h2:file:./data/mymovies;DB_CLOSE_ON_EXIT=FALSE;AUTO_RECONNE
CT=TRUE
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# Configuração JPA / Hibernate
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# Console Web H2 (Acessível em http://localhost:8080/h2-console)
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

1.4. Camada de Serviço e Abstração Spring Data JPA

Com a introdução do Spring Data JPA, a camada de acesso a dados é simplificada drasticamente. Criamos a interface **MovieRepository** estendendo *JpaRepository<Movie, Long>*. O Spring gera dinamicamente em tempo de execução todas as operações CRUD (*save, findById, findAll, deleteById*), eliminando a necessidade de escrever queries SQL manuais.

Dica do Instrutor: A classe **MovieService** passa a ser a responsável por intermediar a regra de negócio e gerir transações com a anotação **@Transactional**. O Controller não conversa diretamente com o Repository, mantendo o desacoplamento de camadas da Clean Architecture.

2. Baseline Git & Estrutura Oficial de Pacotes

Antes de iniciar a alteração de código, garanta que o seu repositório local está no checkpoint oficial da Aula 02. Execute no terminal:

```
// Alinhamento com o Checkpoint da Aula 02
git fetch origin --tags
git checkout checkpoint/aula-02-dtos-validation
```

A nova árvore de diretórios do projeto sob o pacote **pt.rumos.mymovies** ficará organizada da seguinte forma:

```
src/main/java/pt/rumos/mymovies/
├── MyMoviesApplication.java
├── model/
│   └── Movie.java                // Entidade JPA (@Entity, @Table)
├── dto/
│   ├── MovieRequest.java         // DTO de entrada POST/PUT (Bean Validation)
│   ├── MovieResponse.java        // DTO de saída da API (fromDomain)
│   └── RatingUpdateRequest.java  // DTO do PATCH (@DecimalMin/@DecimalMax)
├── repository/
│   └── MovieRepository.java      // Interface JpaRepository<Movie, Long>
├── service/
│   └── MovieService.java         // Camada de Negócio (@Service, @Transactional)
└── controller/
    └── MovieController.java      // Endpoints REST desacoplados
```

3. Roteiro Guiado da Atividade Prática (Passo a Passo)

Etapa 1: Adicionar Dependências no pom.xml

Adicione as dependências do **Spring Data JPA** e do driver **H2 Database** no ficheiro *pom.xml* (ou utilize o atalho Alt+Insert / Edit Starters no IntelliJ):

```
<!-- Spring Data JPA -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- H2 Database -->
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
</dependency>

<!-- H2 Console para visualizar as instrucoes SQL executadas -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-h2console</artifactId>
</dependency>
```

Etapa 2: Converter o Record Movie para Entidade JPA

Refatore a classe **Movie.java** no pacote *pt.rumos.mymovies.model*. Substitua a declaração *record* por uma **class** tradicional com anotações JPA.

```
package pt.rumos.mymovies.model;

import jakarta.persistence.*;

@Entity
@Table(name = "movies")
public class Movie {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "title", nullable = false, length = 120)
    private String title;

    @Column(name = "director", nullable = false)
    private String director;

    @Column(name = "release_year", nullable = false)
    private Integer year;

    @Column(name = "rating")
    private Double rating;

    // TODO: Crie o Construtor Padrão sem argumentos (Exigência do JPA)
    // TODO: Crie o Construtor Completo com parâmetros
    // TODO: Crie os Getters e Setters para todos os atributos
}
```

Etapa 3: Criar o Repositório Spring Data JPA

Crie a interface **MovieRepository.java** no pacote *pt.rumos.mymovies.repository* estendendo *JpaRepository<Movie, Long>*.

```
package pt.rumos.mymovies.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import pt.rumos.mymovies.model.Movie;

public interface MovieRepository extends JpaRepository<Movie, Long> {
    // As operações CRUD básicas (save, findById, findAll, deleteById) são herdadas
automaticamente!
}
```

Etapa 4: Criar a Camada de Serviço (MovieService)

Crie a classe **MovieService.java** no pacote *pt.rumos.mymovies.service*. Esta classe deve ser anotada com **@Service** e conter a anotação **@Transactional(readOnly = true)** nos métodos de leitura.

```
package pt.rumos.mymovies.service;

import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import pt.rumos.mymovies.dto.*;
import pt.rumos.mymovies.model.Movie;
import pt.rumos.mymovies.repository.MovieRepository;
import java.util.List;
import java.util.Optional;

@Service
public class MovieService {

    private final MovieRepository repository;

    public MovieService(MovieRepository repository) {
        this.repository = repository;
    }

    @Transactional(readOnly = true)
    public List<MovieResponse> findAll() {
        // TODO: Buscar todos os filmes do repository e mapear para MovieResponse
    }

    @Transactional(readOnly = true)
    public Optional<MovieResponse> findById(Long id) {
        // TODO: Buscar por id no repository e converter para Optional<MovieResponse>
    }

    @Transactional
    public MovieResponse create(MovieRequest request) {
        // TODO: Converter MovieRequest para a Entidade Movie, salvar no repository e
retornar MovieResponse
    }

    @Transactional
    public Optional<MovieResponse> updateRating(Long id, RatingUpdateRequest request)
{
        // TODO: Buscar filme por id; se existir, atualizar a nota, salvar e retornar
Optional<MovieResponse>
    }

    @Transactional
    public boolean delete(Long id) {
```

```
        // TODO: Verificar se id existe; se sim, eliminar no repository e retornar
true; senão false
    }
}
```

Etapa 5: Refatorar o MovieController

Injete o **MovieService** no **MovieController.java** e delegue a execução dos endpoints REST para os métodos do serviço.

4. DESAFIO EXTRA (Pesquisa Customizada & Prova do Restart)

Para consolidar o aprendizado do Spring Data JPA e comprovar a persistência durável no SGBD H2, execute as duas atividades de desafio abaixo:

Atividade Extra A: Query Method no Repository (Busca por Título)

1. Abra a interface **MovieRepository.java** e adicione o método derivado:

```
List<Movie> findByTitleContainingIgnoreCase(String title);
```

2. Adicione o método de busca por filtro no **MovieService.java** e atualize o método **@GetMapping** do **MovieController** para aceitar o parâmetro opcional *@RequestParam(required = false) String title*.

Atividade Extra B: A Prova do Restart (Persistência no H2 File)

1. Inicie a aplicação no IntelliJ e envie uma requisição **POST /api/movies** via Bruno para cadastrar o filme *"O Padrinho"*.
2. Abra a página **http://localhost:8080/h2-console** no navegador, conecte-se com a JDBC URL *jdbc:h2:file:./data/mymovies* e execute a query **SELECT * FROM MOVIES;**.
3. **Pare a aplicação no IntelliJ (Stop)** e volte a iniciá-la (Run).
4. Execute um **GET /api/movies** no Bruno e confirme que o filme cadastrado permanece salvo no banco de dados!